

Portfolio

Create a Habit Tracking App Finalization Phase/ Phase 3 Abstract

Course: DLBDSOOFPP01 – Object Oriented and Functional Programming with Python

Name: Anita Skynar

Matriculation: 92124577

Submission Date: 26 June 2024

Link to GitHub Repository: https://github.com/SkyFly03/Habit_Tracking_App.git

Project Overview

This project focused on developing a habit tracking application using Python, with an emphasis on object-oriented and functional programming. The app enables users to define, manage, and analyze their habits.

Key Features and Functionalities

1. User Interaction (main.py):

- A command-line interface (CLI) using the questionnaire library for user interaction.
- Features include adding, viewing, analyzing, logging, and deleting habits.

2. Habit Module (habit.py):

- Defines the Habit class with attributes such as name, start_date, log, and periodicity.
- Methods include `__init__()`, `calculate_streak()`, and `log_entry()` to handle habit tasks.

3. Data Management (db.py):

- Stores habit data in JSON format.
- Functions include `load_db()`, `save_db()`, `get_all_habits()`, `get_habit()`, `add_habit()`, and `delete_habit()`.

4. Analytics Module (analyze.py):

- Analyzes habit performance with the `analyze_habits()` function.

Project Directory Structure

- **venv**: Virtual environment containing installed packages and dependencies.
- **analyze.py**: Module for habit analytics.
- **class_diagram.puml**: UML Class diagram visualizing the class structure.
- **db.json**: JSON file for storing habit data.
- **db.py**: Database interaction module.
- **habit.py**: Habit class module.
- **main.py**: Provides CLI for user interaction.

- **preload_db.py**: Script to preload the database with sample data.
- **README.md**: Project documentation.
- **requirements.txt**: Lists of dependencies required. Python packages.
- **test.py**: Unit tests for the application.

What Went Well

- **Tool Selection and Setup**: Choosing tools like PyCharm, Python 3.7+, and setting up a virtual environment.
- **Clear Project Structure**: Organizing the project directory clearly made it easier to manage.
- **User-Friendly CLI**: Using the questionnaire library resulted in a user-friendly interface.

Challenges Faced and Overcome

- **Data Persistence**: Keeping data consistent between user sessions was tough at first. This was tackled by using JSON-based storage, which ensured data stayed reliable.
- **Testing**: Creating thorough unit tests for all features was time-consuming but rewarding, as testing with pytest ensured the reliability and accuracy of all habit tracking and analytics functions.

Additional Value-Added Features

- **Preloaded Data**: The app includes a script (preload_db.py) to load the database with sample data, simplifying initial testing.
- **Detailed Documentation**: The README.md file offers clear instructions for installation, usage, and troubleshooting, making it easy for end-users to navigate.

Conclusion

The developed habit tracking app meets all the requirements and gives users a reliable tool to manage and analyze their habits. It uses object-oriented and functional programming in Python, making it well-structured and easy to use.

By hosting the project on GitHub and including detailed documentation and unit tests, the app is accessible, easy to maintain, and ready for future updates.